

COMS20018, Introduction to AI:

Neural Networks: CNNs, RNNs, and Transformers

Mengyan Zhang

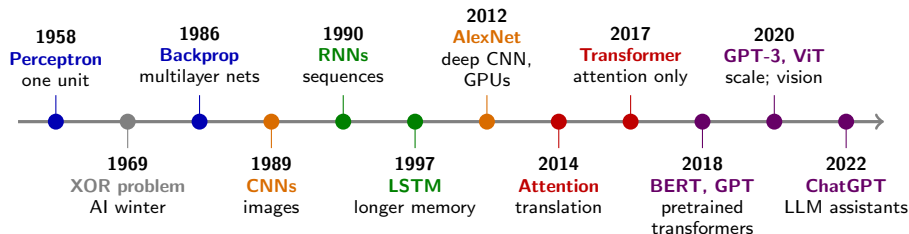
School of Computer Science
University of Bristol

5th October 2026

Agenda

- ▶ A short history, and the different shapes of data
 - ▶ Images: convolutional networks (CNNs)
 - ▶ Sequences: recurrent networks (RNNs), Attention and the Transformer
- ⇒ For each: *what was it built for, what problem did it address, and where is it used?*

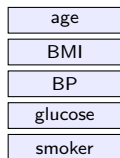
A short history of neural network architectures



- ▶ Each step answered a limitation of the previous one, and the *shape of the data* (grid, sequence) drove the design.

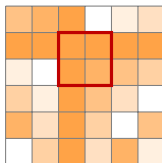
Data comes in different shapes

Vector (tabular)



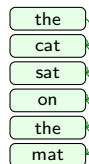
fixed list of features;
their order is arbitrary

Grid (image)



pixels on a 2-D grid;
neighbours are related

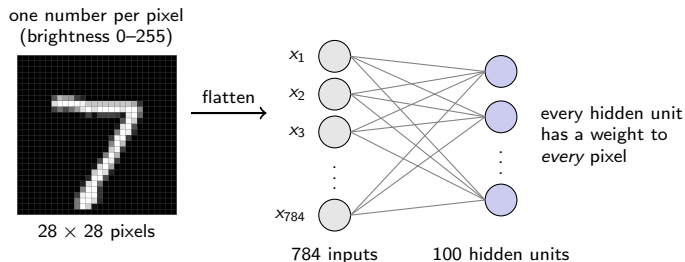
Sequence (text, audio)



ordered, of varying length;
order matters

Shape	Examples	Architecture
vector	patient records, sensor readings	fully connected network
grid	photos, X-rays, satellite images	CNN
sequence	sentences, speech, DNA	RNN/LSTM → Transformer

Images: why a fully connected network struggles

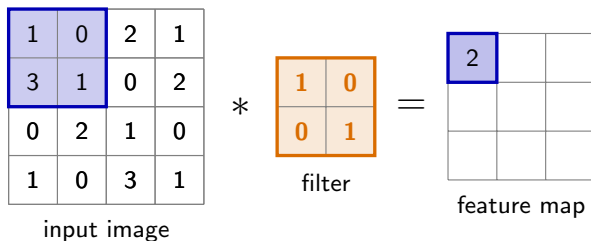


- ▶ **Too many weights.**
 - ▶ **Layout is lost.** Flattening is like putting the photo through a paper shredder: which pixels were neighbours must be re-learned
- ⇒ Two ideas fix both: **local patterns matter** (look at small patches), and a **pattern is the same wherever it appears** (reuse one detector everywhere).

Image: a handwritten “7” from the MNIST dataset (LeCun, Cortes & Burges), CC BY-SA 3.0.

Convolution (1/2): sliding a filter

- **Convolution**: slide a small **filter** (also called a **kernel**) over the image.

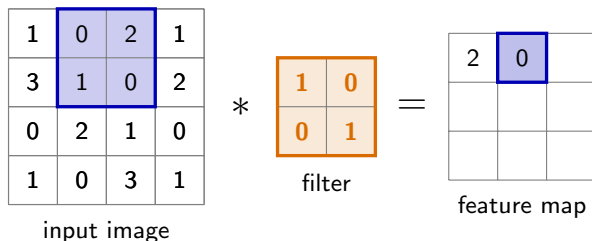


$$1 \cdot 1 + 0 \cdot 0 + 3 \cdot 0 + 1 \cdot 1 = 2$$

- ⇒ The *same* 4 filter weights are reused at every position (**weight sharing**); each output looks only at a small patch (**local receptive field**).
- The filter's numbers are **weights**: they start random and are *learned* by backpropagation, not designed by hand.

Convolution (1/2): sliding a filter

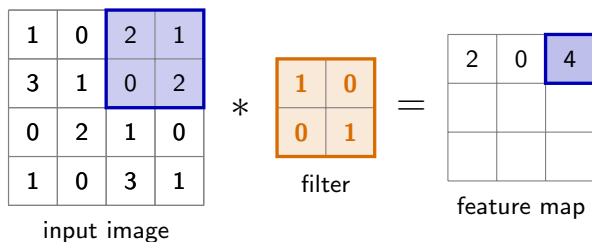
- **Convolution**: slide a small **filter** (also called a **kernel**) over the image.



- ⇒ The *same* 4 filter weights are reused at every position (**weight sharing**); each output looks only at a small patch (**local receptive field**).
- The filter's numbers are **weights**: they start random and are *learned* by backpropagation, not designed by hand.

Convolution (1/2): sliding a filter

- **Convolution**: slide a small **filter** (also called a **kernel**) over the image.



$$2 \cdot 1 + 1 \cdot 0 + 0 \cdot 0 + 2 \cdot 1 = 4$$

- ⇒ The *same* 4 filter weights are reused at every position (**weight sharing**); each output looks only at a small patch (**local receptive field**).
- The filter's numbers are **weights**: they start random and are *learned* by backpropagation, not designed by hand.

Convolution (1/2): sliding a filter

- **Convolution**: slide a small **filter** (also called a **kernel**) over the image.

1	0	2	1
3	1	0	2
0	2	1	0
1	0	3	1

input image

 $*$

1	0
0	1

filter

 $=$

2	0	4
5	2	0
0	5	2

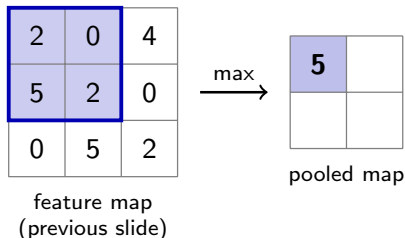
feature map

... and so on, sliding over *every* position

- ⇒ The *same* 4 filter weights are reused at every position (**weight sharing**); each output looks only at a small patch (**local receptive field**).
- The filter's numbers are **weights**: they start random and are *learned* by backpropagation, not designed by hand.

Pooling: shrinking the feature maps

- **Max pooling:** slide a 2×2 window over a feature map and keep only the *largest* value in each window.

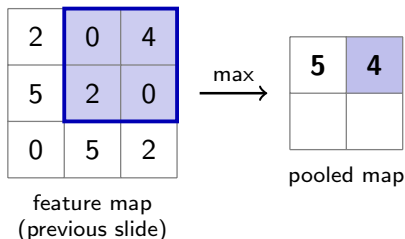


$$\max(2, 0, 5, 2) = 5$$

- *Why?* Smaller maps $\Rightarrow$ less computation in the next layers; and if the pattern moves by a pixel, the maximum barely changes, so the output is **less sensitive to small shifts**.
- No weights to learn: it is a fixed rule.

Pooling: shrinking the feature maps

- **Max pooling:** slide a 2×2 window over a feature map and keep only the *largest* value in each window.

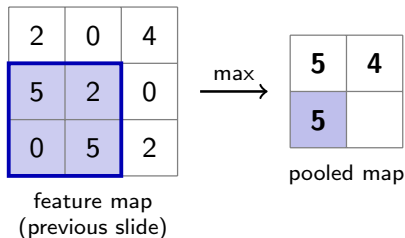


$$\max(0, 4, 2, 0) = 4$$

- *Why?* Smaller maps $\Rightarrow$ less computation in the next layers; and if the pattern moves by a pixel, the maximum barely changes, so the output is **less sensitive to small shifts**.
- No weights to learn: it is a fixed rule.

Pooling: shrinking the feature maps

- **Max pooling:** slide a 2×2 window over a feature map and keep only the *largest* value in each window.

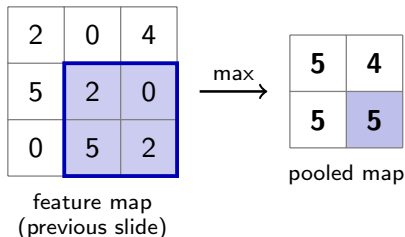


$$\max(5, 2, 0, 5) = 5$$

- *Why?* Smaller maps $\Rightarrow$ less computation in the next layers; and if the pattern moves by a pixel, the maximum barely changes, so the output is **less sensitive to small shifts**.
- No weights to learn: it is a fixed rule.

Pooling: shrinking the feature maps

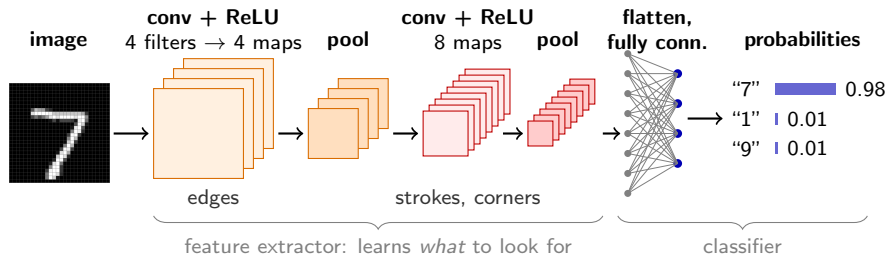
- ▶ **Max pooling:** slide a 2×2 window over a feature map and keep only the *largest* value in each window.



$$\max(2, 0, 5, 2) = 5$$

- ▶ *Why?* Smaller maps $\Rightarrow$ less computation in the next layers; and if the pattern moves by a pixel, the maximum barely changes, so the output is **less sensitive to small shifts**.
- ▶ No weights to learn: it is a fixed rule.

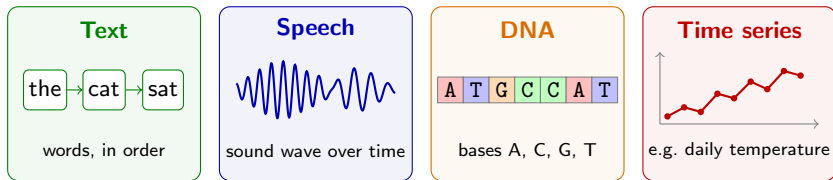
Putting it together: a CNN



- ▶ Each conv layer has **many filters**, one feature map each (vertical edges, curves, textures...); for colour images a filter spans all 3 colour layers. Each map goes through a **ReLU**.
- ▶ Stacking: maps get smaller but more numerous, and features build up from *edges* to *parts* to *objects*.
- ⇒ Used for medical imaging, self-driving cars, face recognition, satellite images... More on **computer vision** in **Week 10**.

Sequences

- ▶ Many kinds of data are not a grid but an **ordered sequence**, of *varying length*:

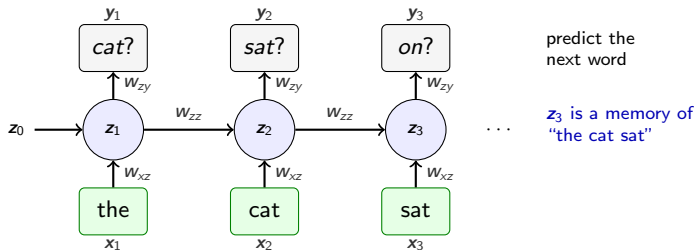


- ▶ **Order matters:** "dog bites man" $\neq$ "man bites dog".
- ⇒ We need an architecture that reads the sequence one element at a time, carrying forward a **memory** of what it's seen so far: a **recurrent neural network (RNN)**.

Recurrent neural networks: the core idea

- ▶ A **hidden state** z_t (a layer of hidden units) is updated from the current input x_t and the previous state:

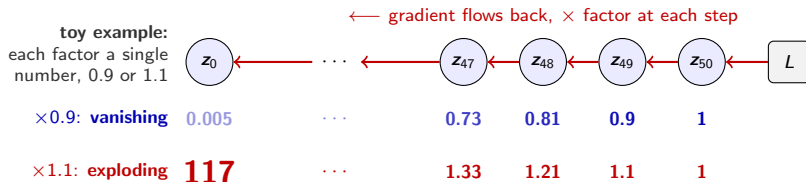
$$z_t = h(W_{zz} z_{t-1} + W_{xz} x_t + b), \quad y_t = f(W_{zy} z_t)$$



- ▶ Read "the cat sat" one word (as a vector x_t) at a time; after each word, guess the next one.
- ▶ The *same* weight matrices W are reused at every step: **weight sharing across time**

RNNs: memory, and its limits

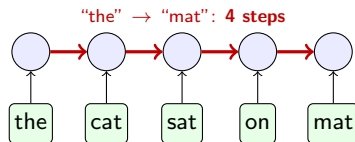
- ▶ RNNs were the state of the art for translation and speech before 2017.
- ▶ But: going back one step, the chain rule **multiplies** the gradient by $\partial \mathbf{z}_t / \partial \mathbf{z}_{t-1}$:



- ▶ **Vanishing gradients**: early words barely affect learning, so long-range links are lost.
- ▶ **Exploding gradients**: huge updates, unstable training.

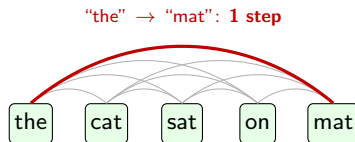
Why move beyond RNNs?

RNN: a long chain



one word after another:
slow, and early words fade

Attention: direct links

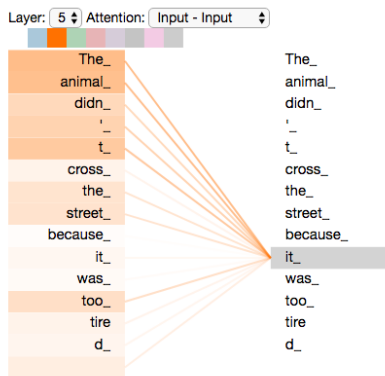


every pair is 1 step apart,
all computed *at once*

- ▶ **Long-range memory:** Vanishing/Exploding gradients. Partial fixes exist, e.g. the **LSTM (Long Short-Term Memory, Hochreiter & Schmidhuber, 1997)**, which adds gates that decide what to keep and what to forget.
 - ▶ **Speed:** z_t needs z_{t-1} *first*, so an RNN can't process the words in parallel; slow to train on GPUs and large datasets.
- ⇒ Idea: let every word look *directly* at every other word, in one step and in parallel: **attention**.

Self-attention: each word looks at every other word

“The animal didn’t cross the street because **it** was too tired.”
What does *it* refer to?



- ▶ *it* **attends** to every word; darker = more attention, mostly to *The animal*.
 - ▶ Its new representation is a **weighted average** of all the words.
- ⇒ Every word, *directly*, in one step.

Figure: J. Alammr, *The Illustrated Transformer* (2018), CC BY-NC-SA 4.0.

The attention mechanism

- ▶ Each word makes three vectors, using learned matrices:
 - ▶ a **query**: what it is looking for;
 - ▶ a **key**: what it offers, matched against others' queries;
 - ▶ a **value**: the information it passes on if matched.
- ▶ Stack them, one row per word, into Q , K , V . Then, for all words at once:

$$\text{Attention}(Q, K, V) = \underbrace{\text{softmax}}_{\text{scores} \rightarrow \text{weights}} \left(\underbrace{\frac{QK^T}{\sqrt{d_k}}}_{\text{scaled scores}} \right) \underbrace{V}_{\text{values}}$$

- ▶ QK^T : **scores** for every (query, key) pair; $\div \sqrt{d_k}$ keeps them moderate.
- ▶ $\times V$: each word's **output** is a weighted average of the **values**.

Softmax: turning scores into weights

Scores $s_1, \dots, s_N \mapsto$ weights $\alpha_m = \exp(s_m) / \sum_{m'} \exp(s_{m'})$: all positive, sum to 1, bigger score $\Rightarrow$ bigger weight. E.g. $(14, 12) \rightarrow (0.88, 0.12)$.

Self-attention step by step

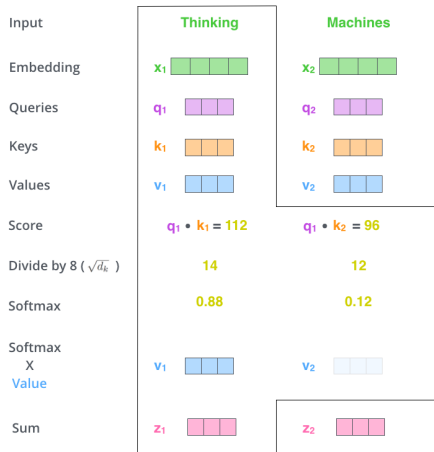


Figure: J. Alamar, *The Illustrated Transformer* (2018), CC BY-NC-SA 4.0.

- By hand, for *Thinking*: **score** (query·key) $\rightarrow \div \sqrt{d_k} \rightarrow$ **softmax** $\rightarrow$ weight the **values** $\rightarrow$ sum.

- In vectors, for word i :

$$\alpha_{ij} = \text{softmax}_j \left(\frac{q_i \cdot k_j}{\sqrt{d_k}} \right)$$

$$z_i = \sum_j \alpha_{ij} v_j$$

- **Attention weights** α_{ij} : how much attention word i pays to each word j (here 0.88, 0.12).
- **Attention output** z_i : the information obtained by combining their **values**:

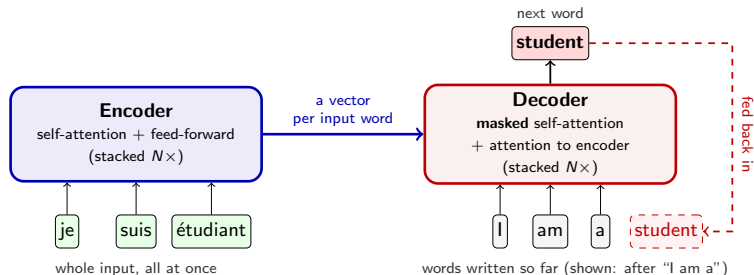
$$z_1 = 0.88 v_1 + 0.12 v_2$$

Multi-head attention, and positional encoding

- ▶ **Multi-head attention:** run several attention computations in parallel, each with its *own* learned $\mathbf{W}_Q$, $\mathbf{W}_K$, $\mathbf{W}_V$, then combine them; one head might learn subject $\leftrightarrow$ pronoun links (as above), another local word order, another something else entirely, all learned rather than specified in advance.
- ▶ **Positional encoding:** attention is a weighted *average* over tokens; on its own, it has **no notion of order** at all. Fix: add a position-dependent vector to each token embedding *before* attention (fixed, e.g. sinusoidal, or learned).

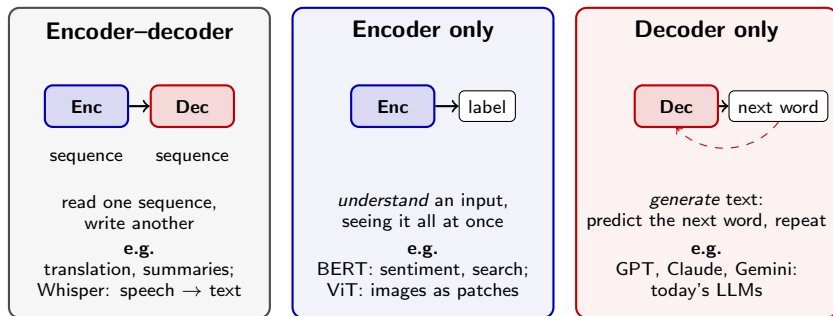
The Transformer: attention is all you need

- **2017** (Vaswani et al.): stack attention blocks into an **encoder** and a **decoder**; Task: French $\rightarrow$ English.



- **Encoder:** reads the whole input at once; a representation of every word, in the context of the others.
- **Decoder:** writes one word per step ("I", "am", "a", "student", *end*), each fed back in; looks at its words so far *and* the encoder.
- The decoder uses **masked (causal) self-attention**: each word attends only to itself and *earlier* words; weights on future words are set to 0 (no peeking ahead).

Transformer variants: encoder, decoder, or both



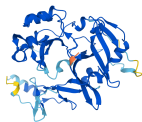
- ▶ Same building block (self-attention + feed-forward) in all three.
- ⇒ Decoder-only models, trained on huge amounts of text to predict the next word, are the large language models (LLMs) behind chatbots.

Where Transformers are used

Explain backprop
in one sentence

It's the chain rule,
applied layer by layer...

Chatbots and LLMs: ChatGPT,
Gemini, Claude

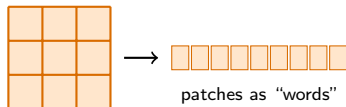


Science: AlphaFold predicts protein
structures

Image: human C12orf29 protein structure predicted by AlphaFold. CSBIOPASSION, Wikimedia Commons, CC BY-SA 4.0. Method: Jumper et al., *Nature* (2021).

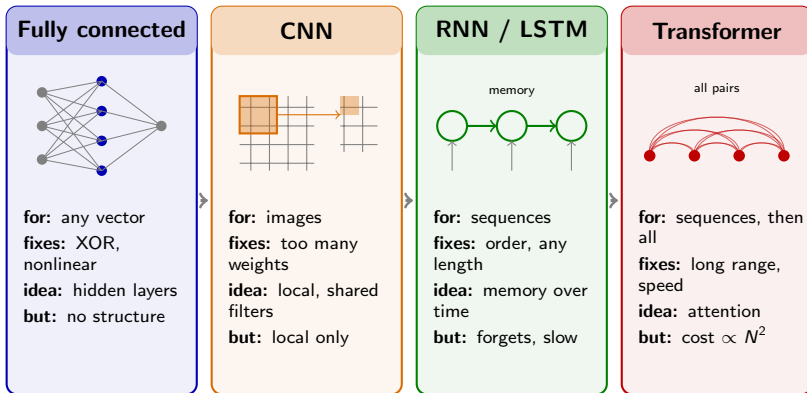
```
def mean(xs):  
  
    return sum(xs)/len(xs)  
  
suggested by the model
```

Code assistants: suggesting the
next lines



Vision (ViT): image classification,
image captioning

The story so far



Reading

- ▶ **Textbook:** Bishop & Bishop, *Deep Learning: Foundations and Concepts* (2024), Ch. 10 (convolutional networks) and Ch. 12 (transformers).
- ▶ **CNNs:** Schovia (S. Ghanta), "How AI Sees: A Visual Guide to Convolutional Neural Networks", YouTube, 12 min, youtube.com/watch?v=n0QkWmOFnjs.
- ▶ **RNNs:** StatQuest (J. Starmer), "Recurrent Neural Networks (RNNs), Clearly Explained!!!", YouTube, 17 min, youtube.com/watch?v=AsNTP8Kwu80
- ▶ **Transformers:** J. Alammr, *The Illustrated Transformer* (2018), jalammar.github.io/illustrated-transformer (highly recommended; source of our attention figures).